

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Обеспечение информационной безопасности мобильных устройств»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Создание приложения и поиск URL, конечных точек и секретов в APK-файлах (Kotlin)»

Выполнили:

Бардышев Артём Антонович,
студент группы N3346



(подпись)

Проверил:

Федоров Иван Романович,
преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Ход работы.....	4
1.1 Работа приложения.....	4
1.2 Получить арк. файл двумя способами: с обфускацией и без обфускации	6
1.3 Декомпилирование и сравнение арк с помощью ArkTool	7
1.4 Проверим приложение через VerusTotal	8
Заключение.....	9

ВВЕДЕНИЕ

Цель работы – изучение безопасности мобильных приложений на платформе Android и получение практических навыков анализа безопасности.

Для достижения поставленной цели необходимо решить следующие задачи:

- ☐ В Android Studio создать проект мобильного приложения на Kotlin;
- ☐ Разработанное приложение должно содержать следующие экраны: экран с выводом ФИО студента; экран, отображающий данные, полученные через вызов к REST API (можно использовать любой доступный публичный REST API); экран с формой обратной связи (можно реализовать взаимодействие с базой данных);
- ☐ Получить apk-файл двумя способами: с обфускацией и без обфускации;
- ☐ Декомпилировать полученные apk с помощью ApkTool, сравнить результаты.
- ☐ Просканировать разработанное приложение утилитой apktool;
- ☐ Просканировать любое стороннее приложение (.apk) утилитой apktool и сравнить результат;
- ☐ Проверить разработанное в рамках ЛР приложение через VirusTotal.

1 ХОД РАБОТЫ

Для начала в среде разработки Android Studio был создан проект мобильного приложения на языке Kotlin. При создании проекта был выбран шаблон Navigation UI Activity, так как он уже содержит заготовку приложения с несколькими экранами и навигацией между ними.

1.1 Работа приложения

После реализации кода было запущено приложение на эмуляторе.

В приложении реализовано три основных экрана:

1. Экран с ФИО;
2. Экран с постами, полученными через API;
3. Экран с формой обратной связи.

Переключение между экранами осуществляется с помощью нижнего меню навигации.

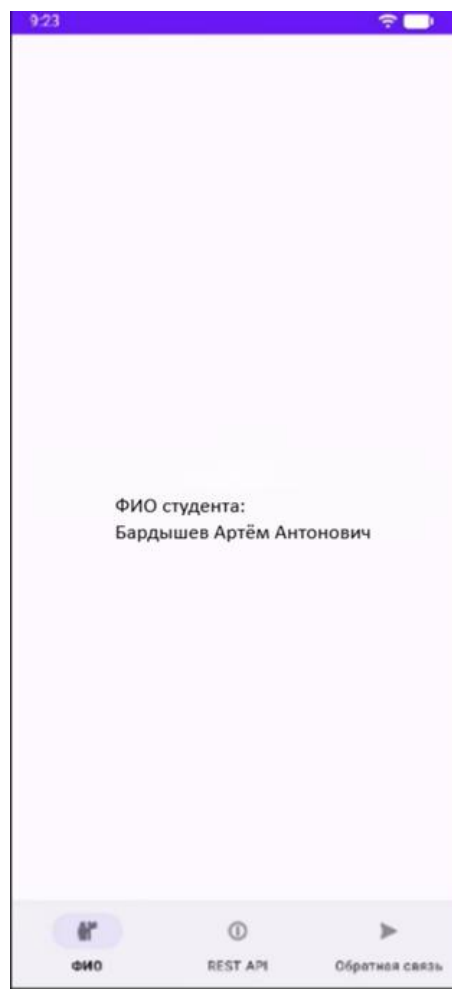


Рисунок 1 – Главный экран



Рисунок 2 – Экран API

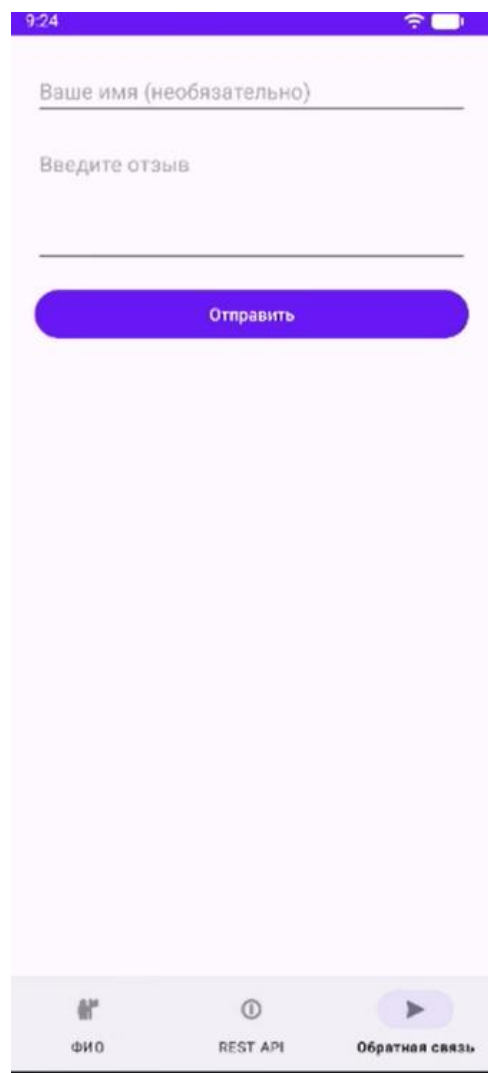


Рисунок 3 – Отзывы

1.2 Получить арк. файл двумя способами: с обфускацией и без обфускации

Теперь получим арк. файл нашего проекта двумя способами: с обфускацией и без обфускации. Обфусцированный код специально делается таким, чтобы затруднить его анализ, понимание алгоритмов работы и модификацию.

Для получения кода без обфускации перейдем по вкладку «Generate APK's». После сборки получим готовый APK файл.

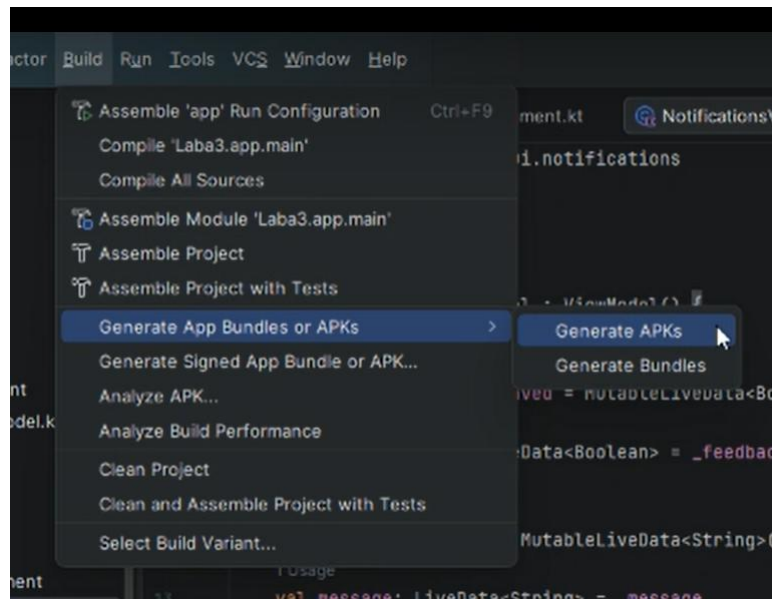


Рисунок 4 – получение арк

Для получения арк файла с обфускацией используем ProGuard. Создадим файл `proguard-rules.pro` и внесем туда список правил, чтобы обфускация не сломала работу APK.

1.3 Декомпилирование и сравнение арк с помощью ArkTool

Декомпилируем полученных APK-файлов с помощью программы ArkTool. И изучим структуру файлов и пространство имен.

Можно заметить, что в версии приложения с обфускацией названия классов и методов были изменены. Это делает код менее понятным и усложняет его анализ.

Теперь перейдем к сравнению логике приложений. Код с обфускацией сложнее читать, и сложнее понимать логику приложения. Также функции имеют другие названия.

Далее APK-файл разработанного приложения необходимо было просканировать с помощью утилиты `arkleaks`.

ArkLeaks — это инструмент, который позволяет находить в APK-файлах различные скрытые данные, например API-ключи, ссылки на серверы и другие возможные конфиденциальные данные.

В результате сканирования обнаружено, что `arkleaks` находит возможные токены, ключи или ссылки, которые используются в приложении. Это является проблемой безопасности, если такие вещи найдут в открытом доступе.

Например, в моем приложении `arkleaks` нашел ссылку на API. Это и является угрозой.

Для проверки также был просканирован apk-файл приложения telegram для компьютера. И сравнен с моим приложением.

В результате обнаружено большое количество сетевых адресов и библиотек, используемых приложением.

1.4 Проверим приложение через VerusTotal

Теперь просканируем созданное приложение с помощью VirusTotal. В итоге увидим, что анализ не нашел никаких подозрительных мест в приложении.

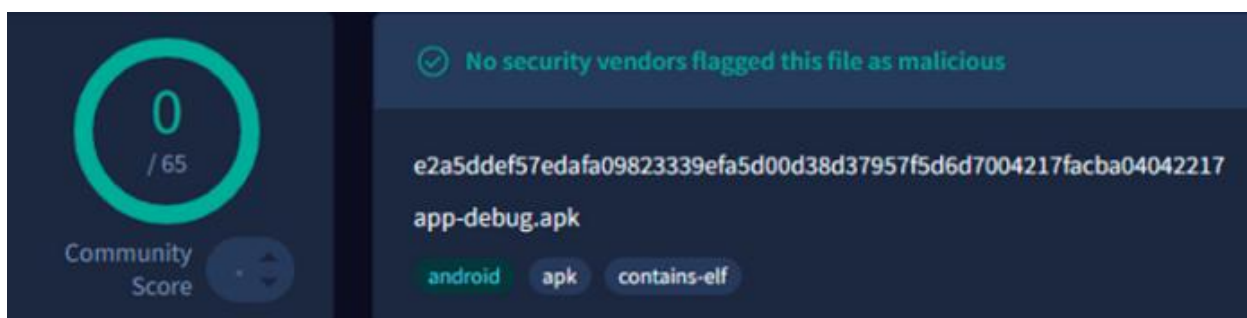


Рисунок 5 – проверка через VirusTotal

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы создано мобильное приложение на Kotlin в среде Android Studio. В приложении реализованы три экрана: экран с ФИО студента, экран с данными из REST API и экран с формой обратной связи. Далее получены APK-файлы двумя способами: с обфускацией и без неё. С помощью ArkTool выполнено декомпилирование файлов и их сравнение. Было замечено, что при обфускации названия классов и функций изменяются, из-за чего код становится сложнее читать. Также приложение проверено с помощью arkleaks, что позволило найти используемые ссылки и данные API. Дополнительно просканировано стороннее приложение для сравнения. В конце APK-файл проверен через VirusTotal, который выполняет анализ файлов на наличие вредоносного кода. В результате проверки подозрительных или вредоносных компонентов в приложении не обнаружено.